图转换器网络

尹成俊，郑敏彬，金来贤，姜在宇*，金炫宇* 韩国高丽大学计算机科学与工程系 {ysj5419, minbyuljeong, raehyun, kangj, hyunwoojkim}@korea.ac.kr

摘要

图神经网络（GNNs）已被广泛应用于图上的表示学习，并在节点分类和链路预测等任务中取得了最先进的性能。然而，大多数现有的GNN是为在固定且同质的图上学习节点表示而设计的。当在错误指定的图或由多种类型的节点和边组成的异质图上学习表示时，这些限制尤其会成为问题。在本文中，我们提出了图变换网络（Graph Transformer Networks, GTNs），该网络能够生成新的图结构，这包括识别原始图中未连接节点之间的有用连接，同时以端到端的方式在新图上学习有效的节点表示。GTNs的核心层——图变换层（Graph Transformer layer），学习边类型和复合关系的软选择，以生成有用的多跳连接，即所谓的元路径。我们的实验表明，GTNs能够学习新的图结构

1 介绍

近年来，图神经网络（GNNs）已被广泛应用于各种图相关任务，如图分类、链接预测和节点分类。研究表明，GNNs 学习到的表示在多个图数据集上（如社交网络、引用网络、大脑功能结构、推荐系统）实现了最先进的性能。GNNs 利用底层图结构，通过将节点特征传递给邻居在图上直接进行卷积，或使用给定图的傅里叶基（即拉普拉斯算子的特征函数）在频域进行卷积。

然而，大多数图神经网络（GNN）的一个局限性是，它们假设用于操作GNN的图结构是固定且同质的。由于上面讨论的图卷积是由固定的图结构决定的，一个存在噪声或缺失/伪造连接的图会导致在图上与错误邻居的无效卷积。此外，在某些应用中，构建用于操作GNN的图并非易事。例如，一个引文网络包含多种类型的节点（如作者、论文、会议）和由它们的关系定义的边（如作者-论文、论文-会议），这被称为异质图。一种简单的做法是忽略节点/边类型，将其视为同质图（具有一种类型的节点和边的标准图）。显然，这是次优的，因为模型无法利用类型信息。一个较新的解决方法是手动设计元路径，这些路径由异质边连接，并进行转换

* 通讯作者

第33届神经信息处理系统大会（NeurIPS 2019），加拿大温哥华。

arXiv:1911.06455v2 [cs.LG] 5 Feb 2020

Graph Transformer Networks

Seongjun Yun, Minbyul Jeong, Raehyun Kim, Jaewoo Kang* , Hyunwoo J. Kim*
Department of Computer Science and Engineering
Korea University
{ysj5419, minbyuljeong, raehyun, kangj, hyunwoojkim}@korea.ac.kr

Abstract

Graph neural networks (GNNs) have been widely used in representation learning on graphs and achieved state-of-the-art performance in tasks such as node classification and link prediction. However, most existing GNNs are designed to learn node representations on the *fixed* and *homogeneous* graphs. The limitations especially become problematic when learning representations on a misspecified graph or a *heterogeneous* graph that consists of various types of nodes and edges. In this paper, we propose Graph Transformer Networks (GTNs) that are capable of generating new graph structures, which involve identifying useful connections between unconnected nodes on the original graph, while learning effective node representation on the new graphs in an end-to-end fashion. Graph Transformer layer, a core layer of GTNs, learns a soft selection of edge types and composite relations for generating useful multi-hop connections so-called *meta-paths*. Our experiments show that GTNs learn new graph structures, based on data and tasks without domain knowledge, and yield powerful node representation via convolution on the new graphs. Without domain-specific graph preprocessing, GTNs achieved the best performance in all three benchmark node classification tasks against the state-of-the-art methods that require pre-defined meta-paths from domain knowledge.

1 Introduction

In recent years, Graph Neural Networks (GNNs) have been widely adopted in various tasks over graphs, such as graph classification [11, 21, 40], link prediction [18, 30, 42] and node classification [3, 14, 33]. The representation learnt by GNNs has been proven to be effective in achieving state-of-the-art performance in a variety of graph datasets such as social networks [7, 14, 35], citation networks [19, 33], functional structure of brains [20], recommender systems [1, 27, 39]. The underlying graph structure is utilized by GNNs to operate convolution directly on graphs by passing node features [12, 14] to neighbors, or perform convolution in the spectral domain using the Fourier basis of a given graph, i.e., eigenfunctions of the Laplacian operator [9, 15, 19].

However, one limitation of most GNNs is that they assume the graph structure to operate GNNs on is *fixed* and *homogeneous*. Since the graph convolutions discussed above are determined by the fixed graph structure, a noisy graph with missing/spurious connections results in ineffective convolution with wrong neighbors on the graph. In addition, in some applications, constructing a graph to operate GNNs is not trivial. For example, a citation network has multiple types of nodes (e.g., authors, papers, conferences) and edges defined by their relations (e.g., author-paper, paper-conference), and it is called a *heterogeneous* graph. A naïve approach is to ignore the node/edge types and treat them as in a *homogeneous* graph (a standard graph with one type of nodes and edges). This, apparently, is suboptimal since models cannot exploit the type information. A more recent remedy is to manually design meta-paths, which are paths connected with heterogeneous edges, and transform

*corresponding author

33rd Conference on Neural Information Processing Systems (NeurIPS 2019), Vancouver, Canada.

将异质图转换为由元路径定义的同质图。然后，传统的图神经网络（GNN）可以在转换后的同质图上进行操作。这是一种两阶段的方法，并且每个问题都需要手工设计元路径。这些元路径的选择会显著影响后续分析的准确性。

在这里，我们开发了图变换网络（Graph Transformer Network, GTN），该网络可以学习将异构输入图转换为每个任务的有用元路径图，并以端到端方式在图上学习节点表示。GTN 可以被看作是空间变换网络（Spatial Transformer Networks）的图形对应物，它能够显式地学习输入图像或特征的空间变换。将异构图转换为由元路径定义的新图结构的主要挑战在于，元路径可能具有任意长度和边类型。例如，文献引用网络中的作者分类可能受益于元路径如作者-论文-作者（APA）或作者-论文-会议-论文-作者（APCPA）。此外，引用网络是有向图，而相对较少的图神经网络能够在其上操作。为了应对这些挑战，我们需要一个模型，该模型能够基于异构图中通过软选择的边类型连接的复合关系生成新的图结构，并且...

我们的贡献如下：(i) 我们提出了一个新颖的框架——图转换网络（Graph Transformer Networks），用于学习一种新的图结构，该结构涉及识别有用的元路径和多跳连接，以在图上学习有效的节点表示。(ii) 图生成具有可解释性，模型能够提供有关预测中有效元路径的洞见。(iii) 我们证明了图转换网络学习的节点表示的有效性，在三个异构图的基准节点分类任务中，其性能优于所有利用额外领域知识的最先进方法。

2 相关工作

图神经网络。近年来，已经开发出许多类别的图神经网络（GNN）用于各种任务。它们可分为两类方法：谱方法和非谱方法。基于谱图理论，Bruna 等人提出了一种利用给定图的傅里叶基在谱域中进行卷积的方法。Kipf 等人通过谱图卷积的一阶近似简化了 GNN。另一方面，非谱方法直接在图上定义卷积操作，利用空间上相邻的节点。例如，Veličković 等人对不同度的节点应用不同的权重矩阵，而 Hamilton 等人提出了可学习的聚合函数，用于总结邻居的信息以进行图表示学习。

使用 GNN 进行节点分类。节点分类已经研究了几十年。传统上，使用了手工制作的特征，例如简单的图统计 [2],、图核 [34]，以及来自局部邻居结构的工程特征 [23]。这些特征不够灵活，性能欠佳。为了克服这一缺点，最近提出了通过图上的随机游走进行节点表示学习的方法，如 DeepWalk [28],、LINE [32], 和 node2vec [13]，借鉴了深度学习模型的技巧（例如 skip-gram），并在性能上有所提升。然而，所有这些方法都仅仅基于图结构来学习节点表示。其表示并没有针对特定任务进行优化。由于 CNN 在表示学习方面取得了显著成功，GNN 可以为给定的任务和数据学习强大的表示。为了提升性能或可扩展性，基于谱卷积的广义卷积 [4, 26],、基于邻居的注意力机制 [25, 33]，subsa

然而，许多现实世界中的问题通常不能仅用单一的同质图来表示。这些图以异质图的形式出现，具有各种类型的节点和边。由于大多数图神经网络（GNN）都是为单一同质图设计的，一种简单的解决方案是采用两阶段方法。利用由多种边类型的复合关系构成的元路径作为预处理手段，将异质图转换为同质图，然后进行表示学习。metapath2vec[10] 通过基于元路径的随机游走学习图表示，而HAN [37] 通过将异质图转化为由元路径构建的同质图来实现图表示学习。然而，这些方法是由领域专家手动选择元路径的，因此可能无法捕捉每个问题的所有有意义关系。而且，性能可能会

a heterogeneous graph into a *homogeneous* graph defined by the meta-paths. Then conventional GNNs can operate on the transformed homogeneous graphs [37, 43]. This is a two-stage approach and requires hand-crafted meta-paths for each problem. The accuracy of downstream analysis can be significantly affected by the choice of these meta-paths.

Here, we develop Graph Transformer Network (GTN) that learns to transform a heterogeneous input graph into useful meta-path graphs for each task and learn node representation on the graphs in an end-to-end fashion. GTNs can be viewed as a graph analogue of Spatial Transformer Networks [16] which explicitly learn spatial transformations of input images or features. The main challenge to transform a heterogeneous graph into new graph structure defined by meta-paths is that meta-paths may have an arbitrary length and edge types. For example, author classification in citation networks may benefit from meta-paths which are Author-Paper-Author (APA) or Author-Paper-Conference-Paper-Author (APCPA). Also, the citation networks are directed graphs where relatively less graph neural networks can operate on. In order to address the challenges, we require a model that generates new graph structures based on composite relations connected with softly chosen edge types in a heterogeneous graph and learns node representations via convolution on the learnt graph structures for a given problem.

Our **contributions** are as follows: (i) We propose a novel framework Graph Transformer Networks, to learn a new graph structure which involves identifying useful meta-paths and multi-hop connections for learning effective node representation on graphs. (ii) The graph generation is interpretable and the model is able to provide insight on effective meta-paths for prediction. (iii) We prove the effectiveness of node representation learnt by Graph Transformer Networks resulting in the best performance against state-of-the-art methods that additionally use domain knowledge in all three benchmark node classification on heterogeneous graphs.

2 Related Works

Graph Neural Networks. In recent years, many classes of GNNs have been developed for a wide range of tasks. They are categorized into two approaches: spectral [5, 9, 15, 19, 22, 38] and non-spectral methods [7, 12, 14, 26, 29, 33]. Based on spectral graph theory, *Bruna et al.* [5] proposed a way to perform convolution in the spectral domain using the Fourier basis of a given graph. *Kipf et al.* [19] simplified GNNs using the first-order approximation of the spectral graph convolution. On the other hand, non-spectral approaches define convolution operations directly on the graph, utilizing spatially close neighbors. For instance, *Veličković et al.* [33] applies different weight matrices for nodes with different degrees and *Hamilton et al.* [14] has proposed learnable aggregator functions which summarize neighbors' information for graph representation learning.

Node classification with GNNs. Node classification has been studied for decades. Conventionally, hand-crafted features have been used such as simple graph statistics [2], graph kernel [34], and engineered features from a local neighbor structure [23]. These features are not flexible and suffer from poor performance. To overcome the drawback, recently node representation learning methods via random walks on graphs have been proposed in DeepWalk [28], LINE [32], and node2vec [13] with tricks from deep learning models (e.g., skip-gram) and have gained some improvement in performance. However, all of these methods learn node representation solely based on the graph structure. The representations are not optimized for a specific task. As CNNs have achieved remarkable success in representation learning, GNNs learn a powerful representation for given tasks and data. To improve performance or scalability, generalized convolution based on spectral convolution [4, 26], attention mechanism on neighbors [25, 33], subsampling [6, 7] and inductive representation for a large graph [14] have been studied. Although these methods show outstanding results, all these methods have a common limitation which only deals with a *homogeneous* graph.

However, many real-world problems often cannot be represented by a single homogeneous graph. The graphs come as a heterogeneous graph with various types of nodes and edges. Since most GNNs are designed for a single homogeneous graph, one simple solution is a two-stage approach. Using meta-paths that are the composite relations of multiple edge types, as a preprocessing, it converts the heterogeneous graph into a homogeneous graph and then learns representation. The metapath2vec [10] learns graph representations by using meta-path based random walk and HAN [37] learns graph representation learning by transforming a heterogeneous graph into a homogeneous graph constructed by meta-paths. However, these approaches manually select meta-paths by domain experts and thus might not be able to capture all meaningful relations for each problem. Also, performance can be

显著受到元路径选择的影响。与这些方法不同，我们的图转换网络可以在异构图上运行，并在学习节点表示的同时端到端地对图进行任务相关的转换。

3 方法

我们框架——图转换网络（Graph Transformer Networks，GTNs）的目标是同时生成新的图结构并在学习到的图上学习节点表示。与大多数假设图已给定的图卷积网络（CNN）不同，GTNs 使用多个候选邻接矩阵来寻找新的图结构，从而执行更高效的图卷积并学习更强大的节点表示。学习新的图结构涉及识别有用的元路径（meta-path），即通过异构边连接的路径，以及多跳连接。在介绍我们的框架之前，我们将简要总结元路径和 GCN 中图卷积的基本概念。

3.1 初步

我们框架的一个输入是多个具有不同类型节点和边的图结构。设$\mathcal{T}^v$ 和 $\mathcal{T}^e$ 分别为节点类型和边类型集合。输入图可以看作异质图 [31] $G=$（V，E），其中 V是节点集合，E是一组观测边，具有节点类型映射函数 f_v： $V \rightarrow \mathcal{T}^v$，边类型映射函数 f_e： $E \rightarrow \mathcal{T}^e$。每个节点 $v_i \in V$都有一个节点类型，即 f_v（v_i） $\in \mathcal{T}^v$。类似地，for $e_{ij} \in E$，f_e（e_{ij}） $\in \mathcal{T}^e$.当$|\mathcal{T}^e|=1$ 和 $|\mathcal{T}^v|=1$ 时，它就变成了一个标准图。本文中，我们考虑 $|\mathcal{T}^e|>1$的情况。设 N 表示节点数，即 $|V|$。异构图可以用一组邻接矩阵 $\{A_k\}_{k=1}^K$ 表示，其中 $K=|\mathcal{T}^e|$，$A_k \in \mathrm{R}^{N \times N}$ 是一个邻接矩阵，其中 $A_k[i,j]$ 在从 j 到 i之间存在第k条类型边时非零。更简洁地说，它可以写成张量 $\mathbb{A} \in \mathrm{R}^{N \times N \times K}$。我们还有一个特征矩阵 $X \in \mathrm{R}^{N \times D}$，意味着每个 的 D 维输入特征

由 p 表示的元路径 **[37]** 是异构图 G 上的一条路径，该路径通过异构边连接，即 $v_1 \xrightarrow{t_1} v_2 \xrightarrow{t_2} \ldots \xrightarrow{t_l} v_{l+1}$，其中 $t_l \in \mathcal{T}^e$ 表示元路径的第 l 种边类型。它定义了节点 v_1 与 v_{l+1}之间的复合关系 $R=t_1 \circ t_2 \ldots \circ t_l$，其中 $R_1 \circ R_2$ 表示关系 R_1 与 R_2 的组合。给定复合关系 R 或边类型序列 $(t_1,t_2,\ldots,t_l)$，元路径 P 的邻接矩阵 $A_\mathcal{P}$ 可通过邻接矩阵的乘法得到。

$$A_\mathcal{P}=A_{t_l}\ldots A_{t_2}A_{t_1}. \tag{1}$$

元路径的概念涵盖了多跳连接，而在我们的框架中，新图结构由邻接矩阵表示。例如，元路径 Author-Paper-Conference（APC），可表示为 $A \xrightarrow{AP} P \xrightarrow{PC} C$，通过 A_{AP} 和 A_{PC} 的相乘生成邻接矩阵 A_{APC}。

图卷积网络（GCN）。在这项工作中，使用图卷积网络（GCN） [19] 以端到端的方式学习节点分类的有用表示。设 $H^{(l)}$ 为GCN中第 l层的特征表示，前向传播过程为

$$H^{(l+1)}=\sigma\left(\tilde{D}^{-\frac{1}{2}}\tilde{A}\tilde{D}^{-\frac{1}{2}}H^{(l)}W^{(l)}\right), \tag{2}$$

其中 $\tilde{A}=A+I \in \mathrm{R}^{N \times N}$ 是加了自连接的图 G 的邻接矩阵 A，$\tilde{D}$ 是 $\tilde{A}$ 的度矩阵，即 $\tilde{D}_{ii}=\sum_i \tilde{A}_{ij}$，而 $W^{(l)} \in \mathrm{R}^{d \times d}$ 是可训练的权重矩阵。人们可以很容易地观察到，跨图的卷积操作是由给定的图结构决定的，除了节点级线性变换 $H^{(l)}W^{(l)}$ 外不可学习。因此，可以将卷积层解释为在节点级线性变换之后，对图进行固定卷积然后应用激活函数 σ 的组合。由于我们学习图结构，我们的框架可从不同卷积中受益，即从学习到的多个邻接矩阵中获得的 $\tilde{D}^{-\frac{1}{2}}\tilde{A}\tilde{D}^{-\frac{1}{2}}$。本节稍后将介绍该架构。对于有向图（即非对称邻接矩阵），式 (2) 中的 A 可以通过入度对角矩阵 D^{-1} 的逆归一化为 $H^{(l+1)}=\sigma(\tilde{D}^{-1}\tilde{A}H^{(l)}W^{(l)})$。

significantly affected by the choice of meta-paths. Unlike these approaches, our Graph Transformer Networks can operate on a heterogeneous graph and transform the graph for tasks while learning node representation on the transformed graphs in an end-to-end fashion.

3 Method

The goal of our framework, Graph Transformer Networks, is to generate new graph structures and learn node representations on the learned graphs simultaneously. Unlike most CNNs on graphs that assume the graph is given, GTNs seek for new graph structures using multiple candidate adjacency matrices to perform more effective graph convolutions and learn more powerful node representations. Learning new graph structures involves identifying useful meta-paths, which are paths connected with heterogeneous edges, and multi-hop connections. Before introducing our framework, we briefly summarize the basic concepts of meta-paths and graph convolution in GCNs.

3.1 Preliminaries

One input to our framework is multiple graph structures with different types of nodes and edges. Let $\mathcal{T}^v$ and $\mathcal{T}^e$ be the set of node types and edge types respectively. The input graphs can be viewed as a heterogeneous graph [31] $G=(V,E)$, where V is a set of nodes, E is a set of observed edges with a node type mapping function $f_v: V \rightarrow \mathcal{T}^v$ and an edge type mapping function $f_e: E \rightarrow \mathcal{T}^e$. Each node $v_i \in V$ has one node type, i.e., $f_v(v_i) \in \mathcal{T}^v$. Similarly, for $e_{ij} \in E$, $f_e(e_{ij}) \in \mathcal{T}^e$. When $|\mathcal{T}^e|=1$ and $|\mathcal{T}^v|=1$, it becomes a standard graph. In this paper, we consider the case of $|\mathcal{T}^e|>1$. Let N denotes the number of nodes, i.e., $|V|$. The heterogeneous graph can be represented by a set of adjacency matrices $\{A_k\}_{k=1}^K$ where $K=|\mathcal{T}^e|$, and $A_k \in \mathbf{R}^{N \times N}$ is an adjacency matrix where $A_k[i,j]$ is non-zero when there is a k-th type edge from j to i. More compactly, it can be written as a tensor $\mathbb{A} \in \mathbf{R}^{N \times N \times K}$. We also have a feature matrix $X \in \mathbf{R}^{N \times D}$ meaning that the D-dimensional input feature given for each node.

Meta-Path [37] denoted by p is a path on the heterogeneous graph G that is connected with heterogeneous edges, i.e., $v_1 \xrightarrow{t_1} v_2 \xrightarrow{t_2} \ldots \xrightarrow{t_l} v_{l+1}$, where $t_l \in \mathcal{T}^e$ denotes an l-th edge type of meta-path. It defines a composite relation $R=t_1 \circ t_2 \ldots \circ t_l$ between node v_1 and v_{l+1}, where $R_1 \circ R_2$ denotes the composition of relation R_1 and R_2. Given the composite relation R or the sequence of edge types $(t_1,t_2,\ldots,t_l)$, the adjacency matrix $A_\mathcal{P}$ of the meta-path P is obtained by the multiplications of adjacency matrices as

$$A_\mathcal{P}=A_{t_l}\ldots A_{t_2}A_{t_1}. \tag{1}$$

The notion of meta-path subsumes multi-hop connections and new graph structures in our framework are represented by adjacency matrices. For example, the meta-path Author-Paper-Conference (APC), which can be represented as $A \xrightarrow{AP} P \xrightarrow{PC} C$, generates an adjacency matrix A_{APC} by the multipication of A_{AP} and A_{PC}.

Graph Convolutional network (GCN). In this work, a graph convolutional network (GCN) [19] is used to learn useful representations for node classification in an end-to-end fashion. Let $H^{(l)}$ be the feature representations of the lth layer in GCNs, the forward propagation becomes

$$H^{(l+1)}=\sigma\left(\tilde{D}^{-\frac{1}{2}}\tilde{A}\tilde{D}^{-\frac{1}{2}}H^{(l)}W^{(l)}\right), \tag{2}$$

where $\tilde{A}=A+I \in \mathbf{R}^{N \times N}$ is the adjacency matrix A of the graph G with added self-connections, $\tilde{D}$ is the degree matrix of $\tilde{A}$, i.e., $\tilde{D}_{ii}=\sum_i \tilde{A}_{ij}$, and $W^{(l)} \in \mathbf{R}^{d \times d}$ is a trainable weight matrix. One can easily observe that the convolution operation across the graph is determined by the given graph structure and it is not learnable except for the node-wise linear transform $H^{(l)}W^{(l)}$. So the convolution layer can be interpreted as the composition of a fixed convolution followed by an activation function σ on the graph after a node-wise linear transformation. Since we learn graph structures, our framework benefits from the different convolutions, namely, $\tilde{D}^{-\frac{1}{2}}\tilde{A}\tilde{D}^{-\frac{1}{2}}$, obtained from learned multiple adjacency matrices. The architecture will be introduced later in this section. For a directed graph (i.e., asymmetric adjacency matrix), $\tilde{A}$ in (2) can be normalized by the inverse of in-degree diagonal matrix D^{-1} as $H^{(l+1)}=\sigma(\tilde{D}^{-1}\tilde{A}H^{(l)}W^{(l)})$.

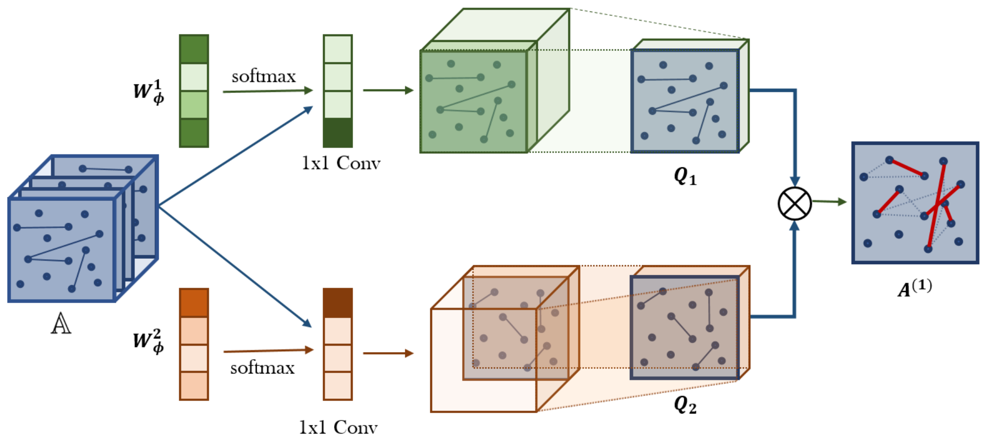


图 1：图变换器层从异构图 G 的邻接矩阵集合 $\mathbb{A}$ 中软选择邻接矩阵（边类型），并通过两个选定邻接矩阵 Q_1 和 Q_2 的矩阵相乘学习由 $A^{(1)}$ 表示的新元路径图。软邻接矩阵选择是候选邻接矩阵的加权和，通过 1×1 卷积与来自 $\mathrm{softmax}(W_\phi^1)$ 的非负权重获得。

3.2 元路径生成

以往的工作 [37, 43] 需要手动定义元路径，并在元路径图上执行图神经网络。相反，我们的图变换网络（GTNs）可以为给定的数据和任务学习元路径，并在学习到的元路径图上进行图卷积。这为发现更有用的元路径提供了机会，并通过使用多个元路径图实现几乎各种各样的图卷积。

图 1 中图变换器（GT）层的新的元路径图生成包含两个部分。首先，GT 层从候选邻接矩阵 $\mathbb{A}$ 中软选择两个图结构 Q_1 和 Q_2。其次，它通过两个关系的组合（即两个邻接矩阵的矩阵乘法，Q_1Q_2）学习一个新的图结构。

它通过如图 1 所示的 1x1 卷积，计算邻接矩阵的凸组合 $\sum_{t_l \in \mathcal{T}^e} \alpha_{t_l}^{(l)} A_{t_l}$，权重来自 softmax 函数，如公式 (4) 所示

$$Q = F(\mathbb{A}; W_\phi) = \phi(\mathbb{A}; \mathrm{softmax}(W_\phi)), \tag{3}$$

ϕ 是卷积层，$W_\phi \in \mathrm{R}^{1\times1\times K}$ 是 ϕ的参数。这个技巧类似于在 [8]中用于低成本图像/动作识别的通道注意力池化。给定两个软选择的邻接矩阵 Q_1 和 Q_2，元路径邻接矩阵通过矩阵乘法Q_1Q_2计算。为了数值稳定性，该矩阵通过它的度矩阵归一化为 $A^{(l)} = D^{-1}Q_1Q_2$。

现在，我们需要检查 GTN 是否能够学习关于边类型和路径长度的任意元路径。任意长度 l 元路径的邻接矩阵可以通过以下方式计算

$$A_P = \left(\sum_{t_1 \in \mathcal{T}^e} \alpha_{t_1}^{(1)} A_{t_1}\right)\left(\sum_{t_2 \in \mathcal{T}^e} \alpha_{t_2}^{(2)} A_{t_2}\right) \ldots \left(\sum_{t_l \in \mathcal{T}^e} \alpha_{t_l}^{(l)} A_{t_l}\right) \tag{4}$$

其中，A_P 表示元路径的邻接矩阵，$\mathcal{T}^e$ 表示一组边类型，$\alpha_{t_l}^{(l)}$ 是第 l层GT中边类型 t_l 的权重。当 α 不是独热向量时，A_P 可以看作是所有长度为l 的元路径邻接矩阵的加权和。因此，一组 l GT层可以像图2所示的GTN架构那样学习任意长度的 l 元路径图结构。该构建方法的一个问题是，增加GT层总是会增加元路径的长度，这不允许保留原始边。在某些应用中，长元路径和短元路径都很重要。为了学习包括原始边在内的短元路径和长元路径，我们引入了单位矩阵

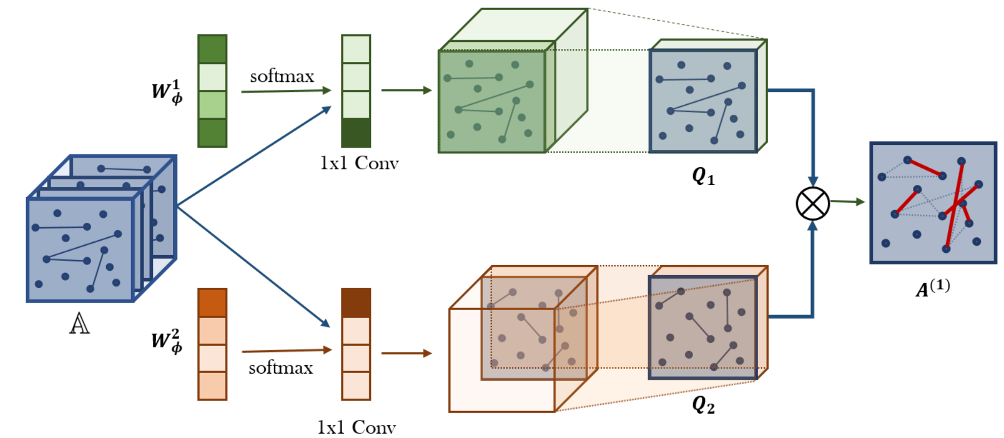


Figure 1: **Graph Transformer Layer** softly selects adjacency matrices (edge types) from the set of adjacency matrices $\mathbb{A}$ of a heterogeneous graph G and learns a new meta-path graph represented by $A^{(1)}$ via the matrix multiplication of two selected adjacency matrices Q_1 and Q_2. The soft adjacency matrix selection is a weighted sum of candidate adjacency matrices obtained by 1×1 convolution with non-negative weights from $\mathrm{softmax}(W_\phi^1)$.

3.2 Meta-Path Generation

Previous works [37, 43] require manually defined meta-paths and perform Graph Neural Networks on the meta-path graphs. Instead, our Graph Transformer Networks (GTNs) learn meta-paths for given data and tasks and operate graph convolution on the learned meta-path graphs. This gives a chance to find more useful meta-paths and lead to virtually various graph convolutions using multiple meta-path graphs.

The new meta-path graph generation in Graph Transformer (GT) Layer in Fig. 1 has two components. First, GT layer softly selects two graph structures Q_1 and Q_2 from candidate adjacency matrices $\mathbb{A}$. Second, it learns a new graph structure by the composition of two relations (i.e., matrix multiplication of two adjacency matrices, Q_1Q_2).

It computes the convex combination of adjacency matrices as $\sum_{t_l \in \mathcal{T}^e} \alpha_{t_l}^{(l)} A_{t_l}$ in (4) by 1x1 convolution as in Fig. 1 with the weights from softmax function as

$$Q = F(\mathbb{A}; W_\phi) = \phi(\mathbb{A}; \mathrm{softmax}(W_\phi)), \tag{3}$$

where ϕ is the convolution layer and $W_\phi \in \mathbf{R}^{1\times1\times K}$ is the parameter of ϕ. This trick is similar to channel attention pooling for low-cost image/action recognition in [8]. Given two softly chosen adjacency matrices Q_1 and Q_2, the meta-path adjacency matrix is computed by matrix multiplication, Q_1Q_2. For numerical stability, the matrix is normalized by its degree matrix as $A^{(l)} = D^{-1}Q_1Q_2$.

Now, we need to check whether GTN can learn an arbitrary meta-path with respect to edge types and path length. The adjacency matrix of arbitrary length l meta-paths can be calculated by

$$A_P = \left(\sum_{t_1 \in \mathcal{T}^e} \alpha_{t_1}^{(1)} A_{t_1}\right)\left(\sum_{t_2 \in \mathcal{T}^e} \alpha_{t_2}^{(2)} A_{t_2}\right) \ldots \left(\sum_{t_l \in \mathcal{T}^e} \alpha_{t_l}^{(l)} A_{t_l}\right) \tag{4}$$

where A_P denotes the adjacency matrix of meta-paths, $\mathcal{T}^e$ denotes a set of edge types and $\alpha_{t_l}^{(l)}$ is the weight for edge type t_l at the lth GT layer. When α is not one-hot vector, A_P can be seen as the weighted sum of all length-l meta-path adjacency matrices. So a stack of l GT layers allows to learn arbitrary length l meta-path graph structures as the architecture of GTN shown in Fig. 2. One issue with this construction is that adding GT layers always increase the length of meta-path and this does not allow the original edges. In some applications, both long meta-paths and short meta-paths are important. To learn short and long meta-paths including original edges, we include the identity

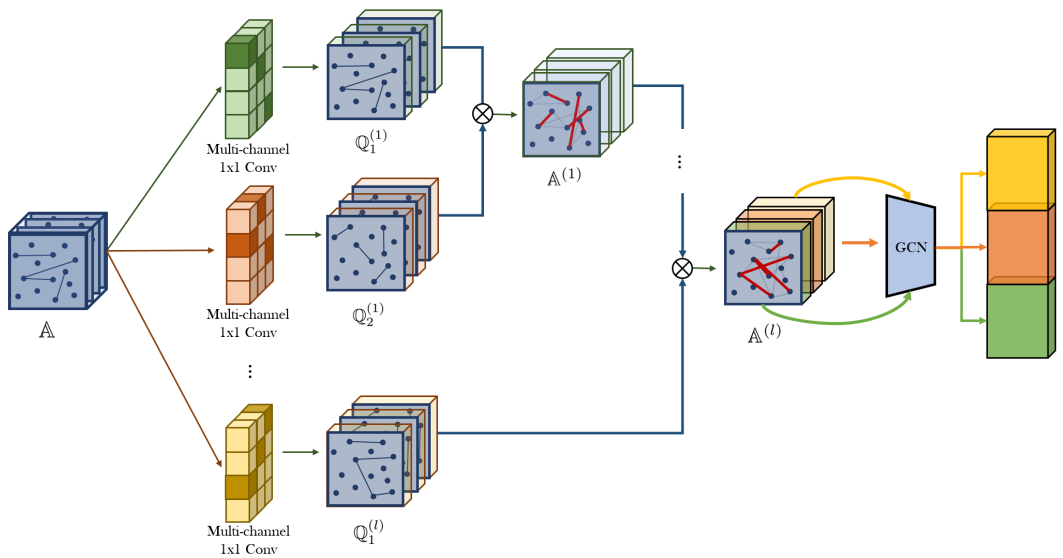


图2：图转换器网络（GTNs）通过GT层学习生成一组新的元路径邻接矩阵，并在新的图结构上像GCN一样执行图卷积。来自同一GCN在多个元路径图上的多重节点表示通过拼接整合，从而提升节点分类的性能。中间邻接张量用于计算第 l层的元路径。

矩阵 I 在 $\mathbb{A}$ 中，即 $A_0 = I$。这个技巧允许 GTN 学习任意长度的元路径，最长可达 $l+1$，当 l 层 GT 层堆叠时。

3.3 图变换器网络

我们在此介绍图变换器网络的架构。为了同时考虑多种类型的元路径，图1中 1×1 卷积的输出通道数设置为 C。然后，GT层产生一组元路径，中间邻接矩阵 Q_1 和 Q_2 变为邻接张量 $\mathbb{Q}_1$ 和 $\mathbb{Q}_2 \in \mathrm{R}^{N\times N\times C}$，如图2所示。通过多种不同的图结构学习不同的节点表示是有益的。在堆叠 l 层GT之后，对元路径张量的每个通道 $\mathbb{A}^{(l)} \in \mathrm{R}^{N\times N\times C}$应用GCN，并将多个节点表示进行连接。

$$Z = \big\|_{i=1}^{C} \sigma(\tilde{D}_i^{-1}\tilde{A}_i^{(l)}XW), \tag{5}$$

其中 $\|$ is 是连接运算符，C 表示通道数量， $\tilde{A}_i^{(l)} = A_i^{(l)} + I$ 是来自 $\mathbb{A}^{(l)}$ 的第 i 个通道的邻接矩阵， $\tilde{D}_i$ 是 $\tilde{A}_i^{(l)}$ 的度矩阵， $W \in \mathrm{R}^{d\times d}$ 是在各通道间共享的可训练权重矩阵， $X \in \mathrm{R}^{N\times d}$ 是特征矩阵。 Z 包含来自 C 个不同元路径图的节点表示，这些元路径长度可变，最多为 $l+1$。它用于节点分类，在其顶部使用两个全连接层，然后是一个 softmax 层。我们的损失函数是对具有真实标签的节点使用标准交叉熵。该架构可以被视为 GT 层学习的多个元路径图上的 GCN 集成模型。

4 个实验

在本节中，我们评估了我们的方法在节点分类任务中相对于多种最先进模型的优势。我们进行了实验和分析，以回答以下研究问题：Q1. GTN生成的新图结构对于节点表示学习是否有效？Q2. GTN能否根据数据集自适应地生成可变长度的元路径？Q3. 我们如何从GTN生成的邻接矩阵中解释每条元路径的重要性？

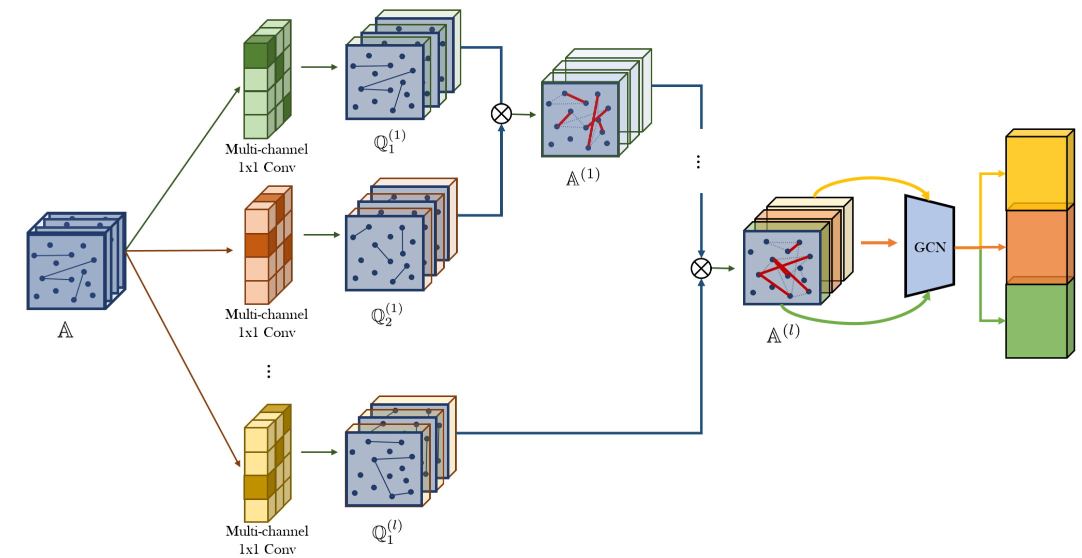


Figure 2: Graph Transformer Networks (GTNs) learn to generate a set of new meta-path adjacency matrices $\mathbb{A}^{(l)}$ using GT layers and perform graph convolution as in GCNs on the new graph structures. Multiple node representations from the same GCNs on multiple meta-path graphs are integrated by concatenation and improve the performance of node classification. $\mathbb{Q}_1^{(l)}$ and $\mathbb{Q}_2^{(l)} \in \mathbf{R}^{N\times N\times C}$ are intermediate adjacency tensors to compute meta-paths at the lth layer.

matrix I in $\mathbb{A}$, i.e., $A_0 = I$. This trick allows GTNs to learn any length of meta-paths up to $l+1$ when l GT layers are stacked.

3.3 Graph Transformer Networks

We here introduce the architecture of Graph Transformer Networks. To consider multiple types of meta-paths simultaneously, the output channels of 1×1 convolution in Fig. 1 is set to C. Then, the GT layer yields a set of meta-paths and the intermediate adjacency matrices Q_1 and Q_2 become adjacency tensors $\mathbb{Q}_1$ and $\mathbb{Q}_2 \in \mathbf{R}^{N\times N\times C}$ as in Fig.2. It is beneficial to learn different node representations via multiple different graph structures. After the stack of l GT layers, a GCN is applied to each channel of meta-path tensor $\mathbb{A}^{(l)} \in \mathbf{R}^{N\times N\times C}$and multiple node representations are concatenated as

$$Z = \big\|_{i=1}^{C} \sigma(\tilde{D}_i^{-1}\tilde{A}_i^{(l)}XW), \tag{5}$$

where $\|$ is the concatenation operator, C denotes the number of channels, $\tilde{A}_i^{(l)} = A_i^{(l)} + I$ is the adjacency matrix from the ith channel of $\mathbb{A}^{(l)}$, $\tilde{D}_i$ is the degree matrix of $\tilde{A}_i^{(l)}$, $W \in \mathbf{R}^{d\times d}$ is a trainable weight matrix shared across channels and $X \in \mathbf{R}^{N\times d}$ is a feature matrix. Z contains the node representations from C different meta-path graphs with variable, at most $l+1$, lengths. It is used for node classification on top and two dense layers followed by a softmax layer are used. Our loss function is a standard cross-entropy on the nodes that have ground truth labels. This architecture can be viewed as an ensemble of GCNs on multiple meta-path graphs learnt by GT layers.

4 Experiments

In this section, we evaluate the benefits of our method against a variety of state-of-the-art models on node classification. We conduct experiments and analysis to answer the following research questions: **Q1.** Are the new graph structures generated by GTN effective for learning node representation? **Q2.** Can GTN adaptively produce a variable length of meta-paths depending on datasets? **Q3.** How can we interpret the importance of each meta-path from the adjacency matrix generated by GTNs?

表 1：异构图节点分类的数据集。

数据集	# 节点	# 边	# 边类型	# 特性	# 训练	# 验证	# 测试
DBLP	18405	67946	4	334	800	400	2857
ACM	8994	25922	4	1902	600	300	2125
IMDB	12772	37288	4	1256	300	300	2339

数据集。为了评估图变换网络生成的元路径的有效性，我们使用了具有多种类型节点和边的异构图数据集。主要任务是节点分类。我们使用了两个引文网络数据集DBLP和ACM，以及一个电影数据集IMDB。实验中使用的异构图的统计信息如表1所示。DBLP包含三种类型的节点（论文（P）、作者（A）、会议（C））、四种类型的边（PA、AP、PC、CP），以及作者的研究领域作为标签。ACM包含三种类型的节点（论文（P）、作者（A）、学科（S））、四种类型的边（PA、AP、PS、SP），以及论文的类别作为标签。两个数据集中的每个节点都表示为关键词的词袋表示。另一方面，IMDB包含三种类型的节点（电影（M）、演员（A）、导演（D）），标签是电影的类型。节点特征以剧情的词袋表示给出。

实现细节。为了公平比较，我们将所有上述方法的嵌入维度设置为64。使用了Adam优化器，并且各个超参数（如学习率、权重衰减等）分别选择，使每个基线方法都能够达到最佳性能。对于基于随机游走的模型，每个节点的游走长度设置为100，迭代次数为1000，窗口大小设置为5，并采样7个负样本。对于GCN、GAT和HAN，参数分别通过验证集进行优化。对于我们的模型GTN，在DBLP和IMDB数据集上使用了三层GT层，在ACM数据集上使用了两层GT层。我们用常数值初始化GT层中的 1×1 卷积层参数。我们的代码已公开，网址为：https://github.com/seongjunyun/Graph_Transformer_Networks。

4.1 基线

为了评估图变换网络（GraphTransformer Networks）在节点分类中学习的表示的有效性，我们将GTN与传统的基于随机游走的基线方法以及最先进的基于图神经网络（GNN）的方法进行比较。

传统的网络嵌入方法已经被研究过，最近 DeepWalk [28] 和 metapath2vec [10] 在基于随机游走的方法中表现出优越的性能。DeepWalk 是一种基于随机游走的网络嵌入方法，最初是为同质图设计的。在这里，我们忽略节点/边的异质性，并在整个异质图上执行DeepWalk。然而，metapath2vec 是一种异质图嵌入方法，它执行基于元路径的随机游走，并利用带负采样的 skip-gram 来生成嵌入。

基于GNN的方法 我们使用了GCN [19]、、GAT [33]，和 HAN [37] 作为基于GNN的方法。GCN是一种图卷积网络，它利用对称图设计的谱图卷积的局部一阶近似。由于我们的数据集是有向图，我们对非对称邻接矩阵修改了度归一化，即 $\tilde{D}^{-1}\tilde{A}$ 而不是 $\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}$。GAT是一种图神经网络，它在同质图上使用注意力机制。我们忽略节点/边的异质性，在整个图上执行GCN和GAT。HAN是一种利用手工选择的元路径的图神经网络。这种方法需要通过预先定义的元路径将原图手动转换为子图。在这里，我们在所选子图上测试HAN，这些子图的节点通过元路径连接，如 [37] 所述。

4.2 节点分类结果

从新图结构中学习到的表示的有效性。表2显示了GTN和其他节点分类基线的性能。通过分析实验结果，我们将回答研究问题Q1和Q2。我们观察到，GTN在所有数据集上相较于所有网络嵌入方法和图神经网络方法都取得了最高的性能。

Table 1: Datasets for node classification on heterogeneous graphs.

Dataset	# Nodes	# Edges	# Edge type	# Features	# Training	# Validation	# Test
DBLP	18405	67946	4	334	800	400	2857
ACM	8994	25922	4	1902	600	300	2125
IMDB	12772	37288	4	1256	300	300	2339

Datasets. To evaluate the effectiveness of meta-paths generated by Graph Transformer Networks, we used heterogeneous graph datasets that have multiple types of nodes and edges. The main task is node classification. We use two citation network datasets DBLP and ACM, and a movie dataset IMDB. The statistics of the heterogeneous graphs used in our experiments are shown in Table 1. DBLP contains three types of nodes (papers (P), authors (A), conferences (C)), four types of edges (PA, AP, PC, CP), and research areas of authors as labels. ACM contains three types of nodes (papers (P), authors (A), subject (S)), four types of edges (PA, AP, PS, SP), and categories of papers as labels. Each node in the two datasets is represented as bag-of-words of keywords. On the other hand, IMDB contains three types of nodes (movies (M), actors (A), and directors (D)) and labels are genres of movies. Node features are given as bag-of-words representations of plots.

Implementation details. We set the embedding dimension to 64 for all the above methods for a fair comparison. The Adam optimizer was used and the hyperparameters (e.g., learning rate, weight decay etc.) are respectively chosen so that each baseline yields its best performance. For random walk based models, a walk length is set to 100 per node for 1000 iterations and the window size is set to 5 with 7 negative samples. For GCN, GAT, and HAN, the parameters are optimized using the validation set, respectively. For our model GTN, we used three GT layers for DBLP and IMDB datasets, two GT layers for ACM dataset. We initialized parameters for 1×1 convolution layers in the GT layer with a constant value. Our code is publicly available at `https://github.com/seongjunyun/Graph_Transformer_Networks`.

4.1 Baselines

To evaluate the effectiveness of representations learnt by the Graph Transformer Networks in node classification, we compare GTNs with conventional random walk based baselines as well as state-of-the-art GNN based methods.

Conventional Network Embedding methods have been studied and recently DeepWalk [28] and metapath2vec [10] have shown predominant performance among random walk based approaches. DeepWalk is a random walk based network embedding method which is originally designed for homogeneous graphs. Here we ignore the heterogeneity of nodes/edges and perform DeepWalk on the whole heterogeneous graph. However, metapath2vec is a heterogeneous graph embedding method which performs meta-path based random walk and utilizes skip-gram with negative sampling to generate embeddings.

GNN-based methods We used the GCN [19], GAT [33], and HAN [37] as GNN based methods. GCN is a graph convolutional network which utilizes a localized first-order approximation of the spectral graph convolution designed for the symmetric graphs. Since our datasets are directed graphs, we modified degree normalization for asymmetric adjacency matrices, i.e., $\tilde{D}^{-1}\tilde{A}$ rather than $\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}$. GAT is a graph neural network which uses the attention mechanism on the homogeneous graphs. We ignore the heterogeneity of node/edges and perform GCN and GAT on the whole graph. HAN is a graph neural network which exploits manually selected meta-paths. This approach requires a manual transformation of the original graph into sub-graphs by connecting vertices with pre-defined meta-paths. Here, we test HAN on the selected sub-graphs whose nodes are linked with meta-paths as described in [37].

4.2 Results on Node Classification

Effectiveness of the representation learnt from new graph structures. Table 2. shows the performances of GTN and other node classification baselines. By analysing the result of our experiment, we will answer the research **Q1** and **Q2**. We observe that our GTN achieves the highest performance on all the datasets against all network embedding methods and graph neural network methods.

表2：节点分类任务的评估结果（F1 分数）。

	深度漫步	metapath2vec	GCN	GAT	HAN	GTN_{-I}	**GTN（拟议）**
DBLP	63.18	85.53	87.30	93.71	92.83	93.91	**94.18**
ACM	67.42	87.61	91.60	92.33	90.96	91.13	**92.68**
IMDB	32.08	35.21	56.89	58.14	56.77	52.33	**60.92**

基于GNN的方法，例如GCN、GAT、HAN和GTN，其表现优于基于随机游走的网络嵌入方法。此外，GAT通常表现优于GCN。这是因为GAT可以为邻居节点指定不同的权重，而GCN只是对邻居节点进行平均。有趣的是，尽管HAN是为异构图修改的GAT，但GAT通常表现优于HAN。这一结果表明，使用 HAN 所采用的预定义元路径可能会对性能产生负面影响。相比之下，我们的GTN模型在所有数据集上都比其他基线模型取得了最佳性能，尽管GTN模型仅使用一层GCN，而GCN、GAT和HAN至少使用两层。这表明GTN能够学习出由对学习更有效的节点表示有用的元路径组成的新图结构。此外，与基线中使用常数的简单元路径邻接矩阵（如HAN）相比，GTN能够

在 $\mathbb{A}$ 中使用单位矩阵来学习可变长度的元路径。如第 3.2 节所述，单位矩阵包含在候选邻接矩阵 $\mathbb{A}$ 中。为了验证单位矩阵的效果，我们训练并评估了另一种名为 GTN_{-I} 的模型作为消融研究。GTN_{-I} 的模型结构与 GTN 完全相同，但其候选邻接矩阵 $\mathbb{A}$ 不包含单位矩阵。总体而言，GTN_{-I} 的表现始终比 GTN 差。值得注意的是，这种差异在 IMDB 中比在 DBLP 中更大。一个解释是，生成的元路径长度 GTN_{-I} 在 IMDB 中并不有效。由于我们堆叠了 3 层 GTL，GTN_{-I} 总是生成长度为 4 的元路径。然而，在 IMDB 中，更短的元路径（例如 MDM）是更理想的。

4.3 图变换器网络的解释

我们研究了GTNs学到的变换，以讨论可解释性问题Q3。我们首先描述如何计算每条元路径在我们的GT层中的重要性。为简化起见，我们假设输出通道的数量为1。为了避免符号上的混乱，我们定义一种简写符号 $\alpha \cdot \mathbb{A} = \sum_k^K \alpha_k A_k$，用于表示输入邻接矩阵的凸组合。图2中第 l 个GT层使用前一层的输出 $A^{(l-1)}$ 和输入邻接矩阵 $\alpha^{(l)} \cdot \mathbb{A}$ 生成一个新元路径图的邻接矩阵 $A^{(l)}$，其计算方式如下：

$$A^{(l)} = \left(D^{(l-1)}\right)^{-1} A^{(l-1)} \left(\sum_i^K \alpha_i^{(l)} A_i\right), \tag{6}$$

其中 $D^{(l)}$ 表示 $A^{(l)}$ 的度矩阵，A_i 表示边类型 i 的输入邻接矩阵，α_i 表示 A_i 的权重。由于在第一层我们有两个凸组合，如图 1 所示，我们记 $\alpha^{(0)} = \text{softmax}(W_\phi^1)$，$\alpha^{(1)} = \text{softmax}(W_\phi^2)$。在我们的 GTN 中，前一张张量的元路径张量会被复用于 $\mathbb{Q}_1^l$，我们只需要每层的 $\alpha^{(l)} = \text{softmax}(W_\phi^2)$ 来计算 $\mathbb{Q}_2^l$。然后，第 l 个 GT 层的新邻接矩阵可以写为

$$A^{(l)} = \left(D^{(l-1)}\right)^{-1} \ldots \left(D^{(1)}\right)^{-1} \left((\alpha^{(0)} \cdot \mathbb{A})(\alpha^{(1)} \cdot \mathbb{A})(\alpha^{(2)} \cdot \mathbb{A}) \ldots (\alpha^{(l)} \cdot \mathbb{A})\right) \tag{7}$$

$$= \left(D^{(l-1)}\right)^{-1} \ldots \left(D^{(1)}\right)^{-1} \left(\sum_{t_0, t_1, \ldots, t_l \in \mathcal{T}^e} \alpha_{t_0}^{(0)} \alpha_{t_1}^{(1)} \ldots \alpha_{t_l}^{(l)} A_{t_0} A_{t_1} \ldots A_{t_l}\right), \tag{8}$$

其中 $\mathcal{T}^e$ 表示一组边类型，$\alpha_{t_l}^{(l)}$ 是第 l 个 GT 层中边类型 t_l 的注意力分数。因此，$A^{(l)}$ 可以被视为包括长度为 1（原始边）到 l 长度的元路径的所有元路径的加权和。元路径 $t_l, t_{l-1}, \ldots, t_0$ 的贡献由 $\prod_{i=0}^l \alpha_{t_i}^{(i)}$ 获得。

Table 2: Evaluation results on the node classification task (F1 score).

	DeepWalk	metapath2vec	GCN	GAT	HAN	GTN_{-I}	**GTN (proposed)**
DBLP	63.18	85.53	87.30	93.71	92.83	93.91	**94.18**
ACM	67.42	87.61	91.60	92.33	90.96	91.13	**92.68**
IMDB	32.08	35.21	56.89	58.14	56.77	52.33	**60.92**

GNN-based methods, *e.g.*, GCN, GAT, HAN, and the GTN perform better than random walk-based network embedding methods. Furthermore, the GAT usually performs better than the GCN. This is because the GAT can specify different weights to neighbor nodes while the GCN simply averages over neighbor nodes. Interestingly, though the HAN is a modified GAT for a heterogeneous graph, the GAT usually performs better than the HAN. This result shows that using the pre-defined meta-paths as the HAN may cause adverse effects on performance. In contrast, Our GTN model achieved the best performance compared to all other baselines on all the datasets even though the GTN model uses only one GCN layer whereas GCN, GAT and HAN use at least two layers. It demonstrates that the GTN can learn a new graph structure which consists of useful meta-paths for learning more effective node representation. Also compared to a simple meta-path adjacency matrix with a constant in the baselines, *e.g.*, HAN, the GTN is capable of assigning variable weights to edges.

Identify matrix in $\mathbb{A}$ to learn variable-length meta-paths. As mentioned in Section 3.2, the identity matrix is included in the candidate adjacency matrices $\mathbb{A}$. To verify the effect of identity matrix, we trained and evaluated another model named GTN_{-I} as an ablation study. the GTN_{-I} has exactly the same model structure as GTN but its candidate adjacency matrix $\mathbb{A}$ doesn't include an identity matrix. In general, the GTN_{-I} consistently performs worse than the GTN. It is worth to note that the difference is greater in IMDB than DBLP. One explanation is that the length of meta-paths GTN_{-I} produced is not effective in IMDB. As we stacked 3 layers of GTL, GTN_{-I} always produce 4-length meta-paths. However shorter meta-paths (e.g. MDM) are preferable in IMDB.

4.3 Interpretation of Graph Transformer Networks

We examine the transformation learnt by GTNs to discuss the question interpretability **Q3**. We first describe how to calculate the importance of each meta-path from our GT layers. For the simplicity, we assume the number of output channels is one. To avoid notational clutter, we define a shorthand notation $\alpha \cdot \mathbb{A} = \sum_k^K \alpha_k A_k$ for a convex combination of input adjacency matrices. The lth GT layer in Fig. 2 generates an adjacency matrix $A^{(l)}$ for a new meta-path graph using the previous layer's output $A^{(l-1)}$ and input adjacency matrices $\alpha^{(l)} \cdot \mathbb{A}$ as follows:

$$A^{(l)} = \left(D^{(l-1)}\right)^{-1} A^{(l-1)} \left(\sum_i^K \alpha_i^{(l)} A_i\right), \tag{6}$$

where $D^{(l)}$ denotes a degree matrix of $A^{(l)}$, A_i denotes the input adjacency matrix for an edge type i and α_i denotes the weight of A_i. Since we have two convex combinations at the first layer as in Fig. 1, we denote $\alpha^{(0)} = \text{softmax}(W_\phi^1)$, $\alpha^{(1)} = \text{softmax}(W_\phi^2)$. In our GTN, the meta-path tensor from the previous tensor is reused for $\mathbb{Q}_1^l$, we only need $\alpha^{(l)} = \text{softmax}(W_\phi^2)$ for each layer to calculate $\mathbb{Q}_2^l$. Then, the new adjacency matrix from the lth GT layer can be written as

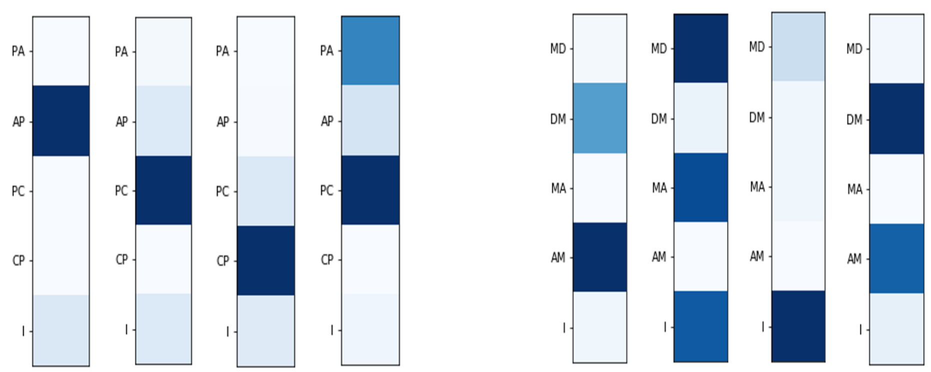
$$A^{(l)} = \left(D^{(l-1)}\right)^{-1} \ldots \left(D^{(1)}\right)^{-1} \left((\alpha^{(0)} \cdot \mathbb{A})(\alpha^{(1)} \cdot \mathbb{A})(\alpha^{(2)} \cdot \mathbb{A}) \ldots (\alpha^{(l)} \cdot \mathbb{A})\right) \tag{7}$$

$$= \left(D^{(l-1)}\right)^{-1} \ldots \left(D^{(1)}\right)^{-1} \left(\sum_{t_0, t_1, \ldots, t_l \in \mathcal{T}^e} \alpha_{t_0}^{(0)} \alpha_{t_1}^{(1)} \ldots \alpha_{t_l}^{(l)} A_{t_0} A_{t_1} \ldots A_{t_l}\right), \tag{8}$$

where $\mathcal{T}^e$ denotes a set of edge types and $\alpha_{t_l}^{(l)}$ is an attention score for edge type t_l at the lth GT layer. So, $A^{(l)}$ can be viewed as a weighted sum of all meta-paths including 1-length (original edges) to l-length meta-paths. The contribution of a meta-path $t_l, t_{l-1}, \ldots, t_0$, is obtained by $\prod_{i=0}^l \alpha_{t_i}^{(i)}$.

表3：与预定义元路径和GTN排在前列的元路径的比较。我们的模型发现了与目标节点（用于节点分类的标签类型节点）之间的预定义元路径一致的重要元路径。同时，GTN还发现了所有类型节点之间新的相关元路径。

数据集	预定义元路径	由 GTN 学习的元路径 目标节点之间的前三名	前三名（全部）
DBLP	APCPA ，APA	APCPA ，阿帕帕，APA	CPCPA ，APCPA ，CP
ACM	PAP, PSP	PAP, PSP	APAP, APA, SPAP
IMDB	MAM ，MDM	MDM, MAM, MDMDM	DM, AM, MDM



(a) (b)

图 3：在图 1 中对 1x1 卷积滤波器 W_ϕ^i (i：层索引) 应用 softmax 函数后，我们可视化了 DBLP（左侧）和 IMDB（右侧）数据集中邻接矩阵（边类型）的注意力分数。(a) 分别表示每条边为 (论文-作者)、(作者-论文)、(论文-会议)、(会议-论文) 以及单位矩阵。(b) IMDB 数据集中的边分别表示 (电影-导演)、(导演-电影)、(电影-演员)、(演员-电影) 以及单位矩阵。

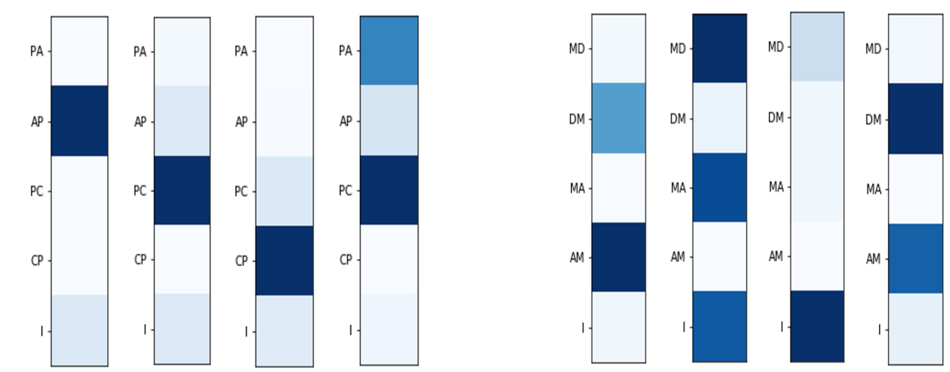
现在我们可以解释 GTNs 学到的新图结构。元路径 $(t_0, t_1, \dots t_l)$ 的权重 $\prod_{i=0}^{l} \alpha_{t_i}^{(i)}$ 是注意力分数，它提供了该元路径在预测任务中的重要性。在表 3 中，我们总结了文献中广泛使用的预定义元路径，以及 GTNs 学到的注意力分数较高的元路径。

如表3所示，在具有预测类别标签的目标节点之间，由领域知识预定义的元路径在GTNs中也始终排名靠前。这表明GTNs能够学习元路径在任务中的重要性。更有趣的是，GTNs还发现了一些不在预定义元路径集合中的重要元路径。例如，在DBLP数据集中，GTN将CPCPA评为最重要的元路径，而它并未包含在预定义的元路径集合中。这是有道理的，因为作者的研究领域（需要预测的标签）与作者发表文章的会议或期刊相关。我们认为，GTNs的可解释性通过元路径的注意力评分为节点分类提供了有用的洞察。

图3显示了来自每个图转换器层的邻接矩阵（边类型）的注意力分数。与DBLP的结果相比，IMDB中的单位矩阵具有更高的注意力分数。如第3.3节所讨论，GTN能够学习比GT层数更短的元路径，而在IMDB中这些路径更有效。通过为单位矩阵分配更高的注意力分数，GTN尝试即使在更深层也坚持较短的元路径。该结果表明，GTN具有根据数据集自适应学习最有效元路径长度的能力。

Table 3: Comparison with predefined meta-paths and top-ranked meta-paths by GTNs. Our model found important meta-paths that are consistent with pre-defined meta-paths between target nodes (a type of nodes with labels for node classifications). Also, new relevant meta-paths between all types of nodes are discovered by GTNs.

Dataset	Predefined Meta-path	Meta-path learnt by GTNs Top 3 (between target nodes)	Top 3 (all)
DBLP	APCPA, APA	APCPA, APAPA, APA	CPCPA, APCPA, CP
ACM	PAP, PSP	PAP, PSP	APAP, APA, SPAP
IMDB	MAM, MDM	MDM, MAM, MDMDM	DM, AM, MDM



(a) (b)

Figure 3: After applying softmax function on 1x1 conv filter W_ϕ^i (i: index of layer) in Figure 1, we visualized this attention score of adjacency matrix (edge type) in DBLP (left) and IMDB (right) datasets. (a) Respectively, each edge indicates (Paper-Author), (Author-Paper), (Paper-Conference), (Conference-Paper), and identity matrix. (b) Edges in IMDB dataset indicates (Movie-Director), (Director-Movie), (Movie-Actor), (Actor-Movie), and identity matrix.

Now we can interpret new graph structures learnt by GTNs. The weight $\prod_{i=0}^{l} \alpha_{t_i}^{(i)}$ for a meta-path $(t_0, t_1, \dots t_l)$ is an attention score and it provides the importance of the meta-path in the prediction task. In Table 3 we summarized predefined meta-paths, that are widely used in literature, and the meta-paths with high attention scores learnt by GTNs.

As shown in Table 3, between target nodes, that have class labels to predict, the predefined meta-paths by domain knowledge are consistently top-ranked by GTNs as well. This shows that GTNs are capable of learning the importance of meta-paths for tasks. More interestingly, GTNs discovered important meta-paths that are not in the predefined meta-path set. For example, in the DBLP dataset GTN ranks CPCPA as most importance meta-paths, which is not included in the predefined meta-path set. It makes sense that author's research area (label to predict) is relevant to the venues where the author publishes. We believe that the interpretability of GTNs provides useful insight in node classification by the attention scores on meta-paths.

Fig.3 shows the attention scores of adjacency matrices (edge type) from each Graph Transformer Layer. Compared to the result of DBLP, identity matrices have higher attention scores in IMDB. As discussed in Section 3.3, a GTN is capable of learning shorter meta-paths than the number of GT layers, which they are more effective as in IMDB. By assigning higher attention scores to the identity matrix, the GTN tries to stick to the shorter meta-paths even in the deeper layer. This result demonstrates that the GTN has ability to adaptively learns most effective meta-path length depending on the dataset.

5 结论

我们提出了图变换网络（Graph Transformer Networks）用于在异构图上学习节点表示。我们的方法将异构图转换为由元路径定义的多个新图，这些元路径具有任意边类型和任意长度，最长可达图变换层数减一，同时通过对学习到的元路径图进行卷积来学习节点表示。学习到的图结构能够带来更有效的节点表示，从而在所有三个异构图基准节点分类任务中实现最先进的性能，而无需任何来自领域知识的预定义元路径。由于我们的图变换层可以与现有的图神经网络（GNNs）结合，我们认为该框架为GNNs自主优化图结构以根据数据和任务进行卷积操作提供了新的途径，无需人工干预。未来有趣的研究方向包括探索将GT层与不同类别的GNNs结合使用的有效性，而不仅仅是与GCNs结合。

6 致谢

本研究得到了韩国国家研究基金会（NRF）的资助，该基金由韩国政府（MSIT）提供（NRF-2019R1G1A1100626, NRF-2016M3A9A7916996, NRF-2017R1A2A1A17069645）。

参考文献

[1] R. v. d. 伯格、T. N. 基普夫和M. 韦林。图卷积矩阵完备。arXiv 预印本 arXiv：1706.02263,2017年。[2] S. 巴加特、G. 科莫德和S. 穆图克里希南。社交网络中的节点分类。载于《社交网络数据分析》，第115–148页。施普林格出版社，2011年。[3] S. 巴加特、G. 科尔莫德和S. 穆图克里希南。社交网络中的节点分类。载于《社交网络数据分析》，第115–148页。2011年。[4] M. M. 布朗斯坦、J. 布鲁纳、Y. 勒昆、A. 斯拉姆和 P. 范德盖恩斯特。几何深度学习：超越欧几里得数据。IEEE信号处理杂志，34（4）：18–42,2017年。[5] J. 布鲁纳、W. 扎伦巴、A. 斯拉姆和Y. 勒昆。谱网络和图上的局部连通网络。arXiv 预印本 arXiv：1312.6203,2013年。[6] 陈志、朱志、宋立。带方差减少的图卷积网络随机训练。arXiv 预印本 arXiv：1710.10568,2017年。[7] 陈志、马、肖志。FastGCN：通过重要性s进行图卷积网络的快速学习

5 Conclusion

We proposed Graph Transformer Networks for learning node representation on a heterogeneous graph. Our approach transforms a heterogeneous graph into multiple new graphs defined by meta-paths with arbitrary edge types and arbitrary length up to one less than the number of Graph Transformer layers while it learns node representation via convolution on the learnt meta-path graphs. The learnt graph structures lead to more effective node representation resulting in state-of-the art performance, without any predefined meta-paths from domain knowledge, on all three benchmark node classification on heterogeneous graphs. Since our Graph Transformer layers can be combined with existing GNNs, we believe that our framework opens up new ways for GNNs to optimize graph structures by themselves to operate convolution depending on data and tasks without any manual efforts. Interesting future directions include studying the efficacy of GT layers combined with different classes of GNNs rather than GCNs. Also, as several heterogeneous graph datasets have been recently studied for other network analysis tasks, such as link prediction [36, 41] and graph classification [17, 24], applying our GTNs to the other tasks can be interesting future directions.

6 Acknowledgement

This work was supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (NRF-2019R1G1A1100626, NRF-2016M3A9A7916996, NRF-2017R1A2A1A17069645).

References

[1] R. v. d. Berg, T. N. Kipf, and M. Welling. Graph convolutional matrix completion. *arXiv preprint arXiv:1706.02263*, 2017.

[2] S. Bhagat, G. Cormode, and S. Muthukrishnan. Node classification in social networks. In *Social network data analytics*, pages 115–148. Springer, 2011.

[3] S. Bhagat, G. Cormode, and S. Muthukrishnan. Node classification in social networks. In *Social network data analytics*, pages 115–148. 2011.

[4] M. M. Bronstein, J. Bruna, Y. LeCun, A. Szlam, and P. Vandergheynst. Geometric deep learning: going beyond euclidean data. *IEEE Signal Processing Magazine*, 34(4):18–42, 2017.

[5] J. Bruna, W. Zaremba, A. Szlam, and Y. LeCun. Spectral networks and locally connected networks on graphs. *arXiv preprint arXiv:1312.6203*, 2013.

[6] J. Chen, J. Zhu, and L. Song. Stochastic training of graph convolutional networks with variance reduction. *arXiv preprint arXiv:1710.10568*, 2017.

[7] J. Chen, T. Ma, and C. Xiao. FastGCN: Fast learning with graph convolutional networks via importance sampling. In *International Conference on Learning Representations*, 2018.

[8] Y. Chen, Y. Kalantidis, J. Li, S. Yan, and J. Feng. Â2-nets: Double attention networks. In *Advances in Neural Information Processing Systems*, pages 352–361, 2018.

[9] M. Defferrard, X. Bresson, and P. Vandergheynst. Convolutional neural networks on graphs with fast localized spectral filtering. In *Advances in neural information processing systems*, pages 3844–3852, 2016.

[10] Y. Dong, N. V. Chawla, and A. Swami. metapath2vec: Scalable representation learning for heterogeneous networks. In *KDD '17*, pages 135–144, 2017.

[11] D. K. Duvenaud, D. Maclaurin, J. Iparraguirre, R. Bombarell, T. Hirzel, A. Aspuru-Guzik, and R. P. Adams. Convolutional networks on graphs for learning molecular fingerprints. In *Advances in neural information processing systems*, pages 2224–2232, 2015.

[12] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl. Neural message passing for quantum chemistry. In *Proceedings of the 34th International Conference on Machine Learning-Volume 70*, pages 1263–1272, 2017.

[13] A. 格罗弗和J. 莱斯科维奇。node2vec：用于网络的可扩展特征学习。载于2016年第22届ACM SIGKDD国际知识发现与数据挖掘会议论文集。[14] W. L. 汉密尔顿、R. 英和J. 莱斯科维茨。大图上的归纳表征学习。CoRR，abs/1706.02216,2017年。[15] M. 亨纳夫、J. 布鲁纳和Y. 勒康。图结构化数据上的深度卷积网络。CoRR，abs/1506.05163,2015年。[16] M. Jaderberg、K. Simonyan、A. Zisserman 等。空间变压器网络。载于《神经信息处理系统进展》，2017–2025页，2015年。[17] 金光、苏光光、郑文、李修、金志和姜。《帽子：股票走势预测的层级图关注网络》，2019年。[18] T. N. 基普夫和M. 韦林。变分图自编码器。NIPS贝叶斯深度学习研讨会，2016年。[19] T. N. 基普夫和M. 韦林。利用图卷积网络进行半监督分类。在国际学习表征会议（ICL）中

[13] A. Grover and J. Leskovec. node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016.

[14] W. L. Hamilton, R. Ying, and J. Leskovec. Inductive representation learning on large graphs. *CoRR*, abs/1706.02216, 2017.

[15] M. Henaff, J. Bruna, and Y. LeCun. Deep convolutional networks on graph-structured data. *CoRR*, abs/1506.05163, 2015.

[16] M. Jaderberg, K. Simonyan, A. Zisserman, et al. Spatial transformer networks. In *Advances in neural information processing systems*, pages 2017–2025, 2015.

[17] R. Kim, C. H. So, M. Jeong, S. Lee, J. Kim, and J. Kang. Hats: A hierarchical graph attention network for stock movement prediction, 2019.

[18] T. N. Kipf and M. Welling. Variational graph auto-encoders. *NIPS Workshop on Bayesian Deep Learning*, 2016.

[19] T. N. Kipf and M. Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017.

[20] S. I. Ktena, S. Parisot, E. Ferrante, M. Rajchl, M. C. H. Lee, B. Glocker, and D. Rueckert. Distance metric learning using graph convolutional networks: Application to functional brain networks. *CoRR*, 2017.

[21] J. Lee, I. Lee, and J. Kang. Self-attention graph pooling. *CoRR*, 2019.

[22] T. Lei, W. Jin, R. Barzilay, and T. Jaakkola. Deriving neural architectures from sequence and graph kernels. In *Proceedings of the 34th International Conference on Machine Learning-Volume 70*, pages 2024–2033, 2017.

[23] D. Liben-Nowell and J. Kleinberg. The link-prediction problem for social networks. *Journal of the American society for information science and technology*, 58(7):1019–1031, 2007.

[24] H. Linmei, T. Yang, C. Shi, H. Ji, and X. Li. Heterogeneous graph attention networks for semi-supervised short text classification. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019.

[25] Z. Liu, C. Chen, L. Li, J. Zhou, X. Li, L. Song, and Y. Qi. Geniepath: Graph neural networks with adaptive receptive paths. *arXiv preprint arXiv:1802.00910*, 2018.

[26] F. Monti, D. Boscaini, J. Masci, E. Rodolà, J. Svoboda, and M. M. Bronstein. Geometric deep learning on graphs and manifolds using mixture model cnns. *CoRR*, abs/1611.08402, 2016.

[27] F. Monti, M. Bronstein, and X. Bresson. Geometric matrix completion with recurrent multi-graph neural networks. In *Advances in Neural Information Processing Systems*, pages 3697–3707, 2017.

[28] B. Perozzi, R. Al-Rfou, and S. Skiena. Deepwalk: Online learning of social representations. In *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 701–710, 2014.

[29] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini. The graph neural network model. *IEEE Transactions on Neural Networks*, 2009.

[30] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. Van Den Berg, I. Titov, and M. Welling. Modeling relational data with graph convolutional networks. In *European Semantic Web Conference*, pages 593–607, 2018.

[31] C. Shi, Y. Li, J. Zhang, Y. Sun, and S. Y. Philip. A survey of heterogeneous information network analysis. *IEEE Transactions on Knowledge and Data Engineering*, 29(1):17–37, 2016.

[32] J. Tang, M. Qu, M. Wang, M. Zhang, J. Yan, and Q. Mei. Line: Large-scale information network embedding. In *WWW*, 2015.

[33] P. Veli ˇckovi ´c， G. Cucurull， A. Casanova， A. Romero， P. Liò 和 Y. Bengio.绘制注意力网络图。载于2018年国际学习表征会议。URL https：// openreview.net/forum?id=rJXMpikCZ.[34] S. V. N. 维什瓦纳坦、N. N. 施劳多尔、R. 孔多尔和K. M. 博格瓦尔特。图核。《机器学习研究杂志》，11（4月）：1201–1242,2010年。[35] 王大、崔摘、朱宇。结构深网络嵌入。载于第22届ACM SIGKDD国际知识发现与数据挖掘会议论文集，第1225–1234页，2016年。[36] X. 王、X. 何、曹永、M. 刘、T.-S.楚。Kgat：知识图谱注意力网络推荐。载于第25届ACM SIGKDD国际知识发现与数据挖掘会议论文集（KDD），第950–958页，2019年。[37] 王旭、纪荣、石志、王明、崔柏、余鹏、叶耀。异构图注意力网络。CoRR，abs/1903.07293,2019年。[38] 徐B、沈浩、曹启、邱永、程晓。图w

[33] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio. Graph attention networks. In *International Conference on Learning Representations*, 2018. URL `https://openreview.net/forum?id=rJXMpikCZ`.

[34] S. V. N. Vishwanathan, N. N. Schraudolph, R. Kondor, and K. M. Borgwardt. Graph kernels. *Journal of Machine Learning Research*, 11(Apr):1201–1242, 2010.

[35] D. Wang, P. Cui, and W. Zhu. Structural deep network embedding. In *Proceedings of the 22Nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1225–1234, 2016.

[36] X. Wang, X. He, Y. Cao, M. Liu, and T.-S. Chua. Kgat: Knowledge graph attention network for recommendation. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*, pages 950–958, 2019.

[37] X. Wang, H. Ji, C. Shi, B. Wang, P. Cui, P. Yu, and Y. Ye. Heterogeneous graph attention network. *CoRR*, abs/1903.07293, 2019.

[38] B. Xu, H. Shen, Q. Cao, Y. Qiu, and X. Cheng. Graph wavelet neural network. In *International Conference on Learning Representations*, 2019.

[39] R. Ying, R. He, K. Chen, P. Eksombatchai, W. L. Hamilton, and J. Leskovec. Graph convolutional neural networks for web-scale recommender systems. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 974–983, 2018.

[40] R. Ying, J. You, C. Morris, X. Ren, W. L. Hamilton, and J. Leskovec. Hierarchical graph representation learning with differentiable pooling. *CoRR*, abs/1806.08804, 2018.

[41] C. Zhang, D. Song, C. Huang, A. Swami, and N. V. Chawla. Heterogeneous graph neural network. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*, pages 793–803, 2019.

[42] M. Zhang and Y. Chen. Link prediction based on graph neural networks. In *Advances in Neural Information Processing Systems*, pages 5165–5175, 2018.

[43] Y. Zhang, Y. Xiong, X. Kong, S. Li, J. Mi, and Y. Zhu. Deep collective classification in heterogeneous information networks. In *Proceedings of the 2018 World Wide Web Conference on World Wide Web*, pages 399–408, 2018.